

Resolucion del Practico 4

Testing de Software 2024

Índice

Introduccion	2
1. Ejercicio 1: Grafo de 4 nodos	2
1.1. (a) NC pero no EC	3
1.2. (b) EC pero no EPC	3
1.3. (c) Conjunto que satisface EPC	3
2. Ejercicio 2: Grafo de 7 nodos	3
2.1. (a) Requisitos de EPC (12)	4
2.2. (b) ¿{p1, p2, p3} satisface EPC?	4
2.3. (c) Tour directo o con sidetrips	4
2.4. (d) Requisitos para NC, EC y PPC	5
2.5. (e) NC pero no EC	6
2.6. (f) EC pero no PPC	6
3. Ejercicio 3: fmtRewrap	6
3.1. Modelo de CFG (a)	6
3.2. (b) Test que sale del <code>while</code> sin entrar al cuerpo	7
3.3. (c) Requisitos NC, EC y PPC	7
3.4. (d) Suite para NC (pero no EC)	9
3.5. (e) Suite para EC pero no PPC	9
3.6. (f) Suite para PPC con Best Effort Touring	10
4. Ejercicio 4: patternIndex – instrumentacion y cobertura	10
4.1. Instrumentacion	10
4.2. Suite JUnit de la tabla del enunciado	11
4.3. Ejecucion	12
4.4. Resultados	12

Introduccion

Este documento resuelve el *Practico 4* de Testing de Software, centrado en el diseno de tests mediante criterios estructurales sobre grafos: cobertura de nodos (NC), cobertura de aristas (EC), cobertura de pares de aristas (EPC) y cobertura de caminos principales (PPC). Los dos primeros ejercicios trabajan sobre grafos abstractos, mientras que los dos ultimos aplican los mismos criterios sobre el Grafo de Flujo de Control (GCF) de los metodos `fmtRerwrap` y `patternIndex`.

A lo largo del documento se utilizan las siguientes convenciones:

- **NC** (Node Coverage): cubrir cada nodo alcanzable.
- **EC** (Edge Coverage): cubrir cada camino de longitud ≤ 1 (cada arista).
- **EPC** (Edge-Pair Coverage): cubrir cada camino de longitud ≤ 2 (cada par de aristas consecutivas).
- **PPC** (Prime Path Coverage): cubrir todos los caminos simples maximales (los que no son subcamino propio de ningun otro camino simple).

1. Ejercicio 1: Grafo de 4 nodos

Se considera el grafo dirigido:

- $N = \{1, 2, 3, 4\}$, $N_0 = \{1\}$, $N_f = \{4\}$.
- $E = \{(1, 2), (2, 3), (3, 2), (2, 4)\}$.

Estructura. Desde 1 solo se puede ir a 2; desde 2 se ramifica a 3 o a 4; desde 3 se vuelve a 2; 4 es el unico nodo final y no tiene aristas salientes. Por consiguiente, todo camino de test valido tiene la forma:

$$1 \rightarrow 2 \rightarrow \underbrace{(\text{ciclo } 2 \leftrightarrow 3 \text{ repetido } k \geq 0 \text{ veces})}_{\text{opcional}} \rightarrow 4.$$

Algunos caminos de test validos: $[1, 2, 4]$, $[1, 2, 3, 2, 4]$, $[1, 2, 3, 2, 3, 2, 4]$, etc.

Requisitos por criterio:

- **NC**: $\{1, 2, 3, 4\}$.
- **EC**: $\{(1, 2), (2, 3), (3, 2), (2, 4)\}$.
- **EPC** (caminos de longitud 2 alcanzables): $[1, 2, 3]$, $[1, 2, 4]$, $[2, 3, 2]$, $[3, 2, 3]$, $[3, 2, 4]$.

1.1. (a) NC pero no EC

No es posible. La topología del grafo fuerza que cualquier conjunto de tests que cubra NC cubra automáticamente EC, por la siguiente razón:

1. Visitar 1 obliga a tomar $(1, 2)$ (única arista saliente de 1).
2. Visitar 3 obliga a tomar $(2, 3)$ (única arista entrante a 3).
3. Una vez en 3, para poder llegar a un final hay que tomar $(3, 2)$ (única arista saliente).
4. Para terminar en 4 hay que usar $(2, 4)$ (única arista entrante a 4).

Los cuatro arcos así forzados son exactamente E , por lo que NC implica EC en este grafo.

1.2. (b) EC pero no EPC

Si es posible. Tomamos $t_1 = [1, 2, 3, 2, 4]$.

- Pares de aristas cubiertos por t_1 : $[1, 2, 3]$, $[2, 3, 2]$, $[3, 2, 4]$.
- Aristas cubiertas: $(1, 2)$, $(2, 3)$, $(3, 2)$, $(2, 4)$, es decir, todo E .

Así, $\{t_1\}$ satisface EC, pero deja sin cubrir $[1, 2, 4]$ y $[3, 2, 3]$, por lo que no satisface EPC.

1.3. (c) Conjunto que satisface EPC

Un conjunto mínimo es:

- $t_1 = [1, 2, 4]$ cubre $[1, 2, 4]$.
- $t_2 = [1, 2, 3, 2, 3, 2, 4]$ cubre $[1, 2, 3]$, $[2, 3, 2]$, $[3, 2, 3]$, $[3, 2, 4]$.

La unión cubre los cinco pares de aristas factibles. No alcanza con un único test porque $[1, 2, 3]$ y $[1, 2, 4]$ son mutuamente excluyentes en el prefijo: una vez que el test elige una de las dos ramas en 2, no puede volver atrás para tomar la otra.

2. Ejercicio 2: Grafo de 7 nodos

Grafo:

- $N = \{1, 2, 3, 4, 5, 6, 7\}$, $N_0 = \{1\}$, $N_f = \{7\}$.
- $E = \{(1, 2), (1, 7), (2, 3), (2, 4), (3, 2), (4, 5), (4, 6), (5, 6), (6, 1)\}$.

Caminos candidatos: $p_1 = [1, 2, 4, 5, 6, 1, 7]$, $p_2 = [1, 2, 3, 2, 4, 6, 1, 7]$, $p_3 = [1, 2, 3, 2, 4, 5, 6, 1, 7]$.

2.1. (a) Requisitos de EPC (12)

Los pares de aristas alcanzables son:

1. $[1, 2, 3]$
2. $[1, 2, 4]$
3. $[2, 3, 2]$
4. $[2, 4, 5]$
5. $[2, 4, 6]$
6. $[3, 2, 3]$
7. $[3, 2, 4]$
8. $[4, 5, 6]$
9. $[4, 6, 1]$
10. $[5, 6, 1]$
11. $[6, 1, 2]$
12. $[6, 1, 7]$

2.2. (b) ¿ $\{p_1, p_2, p_3\}$ satisface EPC?

No. Analizando los pares cubiertos por cada candidato:

- p_1 cubre: $[1, 2, 4], [2, 4, 5], [4, 5, 6], [5, 6, 1], [6, 1, 7]$.
- p_2 cubre: $[1, 2, 3], [2, 3, 2], [3, 2, 4], [2, 4, 6], [4, 6, 1], [6, 1, 7]$.
- p_3 cubre: $[1, 2, 3], [2, 3, 2], [3, 2, 4], [2, 4, 5], [4, 5, 6], [5, 6, 1], [6, 1, 7]$.

Union: $\{[1, 2, 3], [1, 2, 4], [2, 3, 2], [2, 4, 5], [2, 4, 6], [3, 2, 4], [4, 5, 6], [4, 6, 1], [5, 6, 1], [6, 1, 7]\}$.

Pares faltantes: $[3, 2, 3]$ y $[6, 1, 2]$. Ninguno de los tres caminos repite el ciclo $2 \rightarrow 3 \rightarrow 2$ ni retoma el ciclo externo despues de volver a 1.

2.3. (c) Tour directo o con sidetrips

Camino simple $q = [3, 2, 4, 5, 6]$; camino de test $t = [1, 2, 3, 2, 4, 6, 1, 2, 4, 5, 6, 1, 7]$.

No es tour directo: la primera vez que t llega a 4 toma $4 \rightarrow 6$, mientras que q exige $4 \rightarrow 5$. Por lo tanto las aristas de q no aparecen *en el mismo orden* sin interrupciones.

Si es tour con sidetrip. La idea de un sidetrip es: desde un nodo del camino simple, el camino de test se desvia y vuelve al *mismo* nodo del camino simple. Concretamente:

- t pasa por 3, luego por 2, luego por 4 (tres primeros nodos de q).
- Desde 4, t se desvia tomando $[4, 6, 1, 2, 4]$ y vuelve al mismo nodo 4.
- Recien entonces continua con $4 \rightarrow 5 \rightarrow 6$, completando los dos nodos finales de q .

Sidetrip: $[4, 6, 1, 2, 4]$.

2.4. (d) Requisitos para NC, EC y PPC

NC: $\{1, 2, 3, 4, 5, 6, 7\}$.

EC: las nueve aristas $(1, 2), (1, 7), (2, 3), (2, 4), (3, 2), (4, 5), (4, 6), (5, 6), (6, 1)$.

PPC (caminos principales). Un camino principal es un camino simple maximal. Los caminos simples relevantes (no extensibles a otro camino simple) son:

1. $[1, 2, 4, 6, 1]$
2. $[1, 2, 4, 5, 6, 1]$
3. $[2, 3, 2]$
4. $[2, 4, 6, 1, 2]$
5. $[2, 4, 5, 6, 1, 2]$
6. $[3, 2, 3]$
7. $[3, 2, 4, 6, 1, 7]$
8. $[3, 2, 4, 5, 6, 1, 7]$
9. $[4, 6, 1, 2, 3]$
10. $[4, 6, 1, 2, 4]$
11. $[4, 5, 6, 1, 2, 3]$
12. $[4, 5, 6, 1, 2, 4]$
13. $[5, 6, 1, 2, 4, 5]$
14. $[6, 1, 2, 4, 6]$
15. $[6, 1, 2, 4, 5, 6]$

2.5. (e) NC pero no EC

Posible con $t = [1, 2, 3, 2, 4, 5, 6, 1, 7]$: visita los siete nodos, pero no recorre (4, 6) (al llegar a 4 toma la rama hacia 5). Por lo tanto cumple NC pero falla EC.

2.6. (f) EC pero no PPC

Posible con $\{t_1, t_2\}$ donde $t_1 = [1, 2, 4, 5, 6, 1, 7]$ y $t_2 = [1, 2, 3, 2, 4, 6, 1, 7]$.

- Aristas cubiertas: (1, 2), (2, 4), (4, 5), (5, 6), (6, 1), (1, 7) (de t_1) y (2, 3), (3, 2), (4, 6) (de t_2). Las nueve aristas están cubiertas.
- Pero el prime path [3, 2, 3] no aparece: ningún test repite el ciclo $2 \rightarrow 3 \rightarrow 2 \rightarrow 3$. Tampoco aparece [6, 1, 2, 4, 6] (no se ejecuta el ciclo externo dos veces consecutivas), entre otros.

Como queda al menos un prime path sin cubrir, el conjunto satisface EC pero no PPC.

3. Ejercicio 3: fmtRewrap

Se trabaja sobre el método `fmtRewrap(String S, int N)` de `assignment4_exercises.FmtRewrap.java`. Para mantener el análisis manejable se utiliza un CFG *por bloques* agrupando las sentencias lógicas del cuerpo del `while`.

3.1. Modelo de CFG (a)

Nodos del modelo.

- **W**: condición del `while` (`i < S.length()`).
- **C**: clasificación del carácter actual (cadena `if/else if` que asigna `state`).
- **BW**: caso `betweenWord` dentro del `switch`.
- **LB**: caso `lineBreak` dentro del `switch`.
- **CR?**: caso `crFound` (decide entre `hard-CR` y `soft-CR`).
- **CRh**: rama `hard-CR` (`i+1 < S.length() && SArr[i+1] == CR`).
- **CRs**: rama `soft-CR` (rama `else` del caso anterior).
- **IW**: caso `inWord` o por defecto.
- **INC**: incremento `i++`.
- **EXIT**: salida del `while` y `return`.

Aristas:

$W \rightarrow C$ (true), $W \rightarrow \text{EXIT}$ (false),
 $C \rightarrow BW$, $C \rightarrow LB$, $C \rightarrow CR?$, $C \rightarrow IW$,
 $BW \rightarrow INC$, $LB \rightarrow INC$,
 $CR? \rightarrow CRh$, $CR? \rightarrow CRs$,
 $CRh \rightarrow INC$, $CRs \rightarrow INC$, $IW \rightarrow INC$,
 $INC \rightarrow W$.

3.2. (b) Test que sale del while sin entrar al cuerpo

Caso: $t = (S = , N = 10)$.

Cuando S es la cadena vacía, $S.length() == 0$ y la guarda del while $i < S.length()$ es 0
 $< 0 = \text{false}$. El flujo toma directamente $W \rightarrow \text{EXIT}$ y ejecuta $S = \text{new String(SArr)}$
 $+ CR; \text{return}$, devolviendo $\backslash n$.

3.3. (c) Requisitos NC, EC y PPC

NC: $\{W, C, BW, LB, CR?, CRh, CRs, IW, INC, EXIT\}$ (10 requisitos).

EC: las 14 aristas listadas en el CFG.

PPC: sobre este CFG reducido se obtienen 47 prime paths, agrupados por su nodo inicial:

1. $[W, C, BW, INC, W]$
2. $[W, C, LB, INC, W]$
3. $[W, C, IW, INC, W]$
4. $[W, C, CR?, CRh, INC, W]$
5. $[W, C, CR?, CRs, INC, W]$
6. $[C, BW, INC, W, C]$
7. $[C, BW, INC, W, EXIT]$
8. $[C, LB, INC, W, C]$
9. $[C, LB, INC, W, EXIT]$
10. $[C, IW, INC, W, C]$
11. $[C, IW, INC, W, EXIT]$
12. $[C, CR?, CRh, INC, W, C]$

13. $[C, CR?, CRh, INC, W, EXIT]$
14. $[C, CR?, CRs, INC, W, C]$
15. $[C, CR?, CRs, INC, W, EXIT]$
16. $[BW, INC, W, C, BW]$
17. $[BW, INC, W, C, LB]$
18. $[BW, INC, W, C, IW]$
19. $[BW, INC, W, C, CR?, CRh]$
20. $[BW, INC, W, C, CR?, CRs]$
21. $[LB, INC, W, C, BW]$
22. $[LB, INC, W, C, LB]$
23. $[LB, INC, W, C, IW]$
24. $[LB, INC, W, C, CR?, CRh]$
25. $[LB, INC, W, C, CR?, CRs]$
26. $[CR?, CRh, INC, W, C, BW]$
27. $[CR?, CRh, INC, W, C, LB]$
28. $[CR?, CRh, INC, W, C, CR?]$
29. $[CR?, CRh, INC, W, C, IW]$
30. $[CR?, CRs, INC, W, C, BW]$
31. $[CR?, CRs, INC, W, C, LB]$
32. $[CR?, CRs, INC, W, C, CR?]$
33. $[CR?, CRs, INC, W, C, IW]$
34. $[CRh, INC, W, C, CR?, CRh]$
35. $[CRh, INC, W, C, CR?, CRs]$
36. $[CRs, INC, W, C, CR?, CRh]$
37. $[CRs, INC, W, C, CR?, CRs]$
38. $[IW, INC, W, C, BW]$

- 39. $[IW, INC, W, C, LB]$
- 40. $[IW, INC, W, C, IW]$
- 41. $[IW, INC, W, C, CR?, CRh]$
- 42. $[IW, INC, W, C, CR?, CRs]$
- 43. $[INC, W, C, BW, INC]$
- 44. $[INC, W, C, LB, INC]$
- 45. $[INC, W, C, IW, INC]$
- 46. $[INC, W, C, CR?, CRh, INC]$
- 47. $[INC, W, C, CR?, CRs, INC]$

3.4. (d) Suite para NC (pero no EC)

Suite: `FmtRewrapNodeCoverageTest.java`, ejecutable con

```

JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -Dmaven.repo.local
=.m2 \
-Dtest=FmtRewrapNodeCoverageTest test

```

JaCoCo reporta sobre `fmtRewrap`: LINE 29/29, BRANCH 16/16.

Observacion. Para esta implementacion y este nivel de modelado, cubrir todas las sentencias termina cubriendo tambien todas las ramas que la herramienta mide. En la practica, *no* se logro armar una suite que satisfaga NC sin satisfacer EC al mismo tiempo segun JaCoCo. La razon es que muchas sentencias y ramas estan tan acopladas que llegar a un nodo del CFG fuerza la ejecucion de su arista saliente unica.

3.5. (e) Suite para EC pero no PPC

Suite: `FmtRewrapEdgeButNotPrimePathCoverageTest.java`. JaCoCo reporta nuevamente LINE 29/29 y BRANCH 16/16.

Por que la suite no satisface PPC: no se recorre, por ejemplo, el prime path $[BW, INC, W, C, IW]$. Intuitivamente, cada vez que se entra en **BW** (caso `betweenWord`), la siguiente iteracion “util” del bucle pasa por **LB** o por otra rama, no por **IW**, por lo que esa secuencia particular nunca aparece en los caminos ejecutados.

3.6. (f) Suite para PPC con Best Effort Touring

Suite: `FmtRewrapPrimePathBestEffortCoverageTest.java`. JaCoCo: LINE 29/29, BRANCH 16/16.

Cobertura sobre el CFG modelado.

- Prime paths totales: 47.
- Cubiertos directamente: 43.
- No cubiertos: 4. Todos *infactibles* por la semantica del metodo:
 1. $[CR?, CRh, INC, W, C, LB]$
 2. $[CR?, CRs, INC, W, C, CR?]$
 3. $[CRs, INC, W, C, CR?, CRh]$
 4. $[CRs, INC, W, C, CR?, CRs]$

Justificacion de la infactibilidad:

- **Hard-CR seguido de LB.** Despues de tomar CRh, el codigo fija `col = 1`. Para que en la siguiente iteracion se entre a LB seria necesario que `col >= N`, pero N debiera ser tan pequeno que la iteracion previa ya habria disparado LB y no se habria podido tomar CRh. Las dos condiciones son mutuamente excluyentes.
- **Soft-CR seguido de otro CR?.** CRs se elige justamente cuando el caracter siguiente *no* es CR. En la iteracion siguiente, el caracter actual ya no es CR, por lo que no se puede volver a entrar a CR? en la posicion inmediata.

Conclusion. La suite cubre todos los prime paths factibles del CFG bajo *Best Effort Touring* y mantiene cobertura total de lineas y ramas. PPC subsume EC, por lo que la cobertura de aristas tambien queda garantizada.

4. Ejercicio 4: `patternIndex` – instrumentacion y cobertura

Se trabaja sobre `patternIndex(String subject, String pattern)` en `assignment4_exercises.Pat`

4.1. Instrumentacion

Para registrar los caminos efectivamente ejecutados, se inserto una llamada de tipo `PatternIndexPathTracker.hit("X")` en cada nodo del CFG. La clase auxiliar `PatternIndexPathTrack`

esta en `src/main/java/assignment4_exercises/PatternIndexPathTracker.java` y mantiene, por invocacion, la secuencia de nodos visitados.

Nodos instrumentados:

- **S**: inicio del metodo.
- **W**: condicion del `while` externo.
- **I**: comparacion del primer caracter del patron.
- **M**: bloque de inicializacion cuando coincide el primer caracter.
- **FT**: condicion verdadera del `for` interno.
- **IFM**: chequeo de mismatch dentro del `for`.
- **MIS**: bloque de mismatch seguido de `break`.
- **FI**: avance del `for` cuando no hubo mismatch.
- **FF**: terminacion del `for` sin `break`.
- **INC**: incremento `iSub++`.
- **R**: `return`.

Aristas del CFG instrumentado:

$$\begin{aligned}
S &\rightarrow W, & W &\rightarrow I, & W &\rightarrow R, \\
I &\rightarrow M, & I &\rightarrow INC, \\
M &\rightarrow FT, & M &\rightarrow FF, \\
FT &\rightarrow IFM, & IFM &\rightarrow MIS, & IFM &\rightarrow FI, \\
FI &\rightarrow FT, & FI &\rightarrow FF, \\
MIS &\rightarrow INC, & FF &\rightarrow INC, & INC &\rightarrow W.
\end{aligned}$$

En total 15 aristas.

4.2. Suite JUnit de la tabla del enunciado

La suite `PatternIndexInstrumentationCoverageTest.java` ejecuta los 10 casos pedidos:

subject	pattern	output
<code>.a"</code>	<code>"bc"</code>	<code>-1</code>
<code>.ab"</code>	<code>.a"</code>	<code>0</code>
<code>.ab"</code>	<code>.ab"</code>	<code>0</code>

subject	pattern	output
. ^a b"	. ^a c"	-1
. ^a b"	"b"	1
. ^a b"	ç"	-1
. ^a bc"	. ^a bc"	0
. ^a bc"	. ^a bd"	-1
. ^a bc"	"ba"	-1
. ^a bc"	"bc"	1

Al finalizar la suite, un metodo `@AfterClass` consulta el tracker, calcula automaticamente los caminos principales del CFG por DFS y genera un reporte con:

- secuencia de nodos por cada invocacion,
- cobertura de aristas (cubiertas / total),
- cobertura de caminos principales (cubiertos / total),
- lista de requisitos cubiertos y faltantes por criterio.

Ubicacion del reporte: `tp4/assignmnet-4-rodeghiero/target/patternindex-path-report.txt`.

4.3. Ejecucion

```

JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -Dmaven.repo.local
=.m2 \
-Dtest=PatternIndexInstrumentationCoverageTest test

```

4.4. Resultados

Sobre el CFG instrumentado y con los 10 casos de la tabla:

- **Cobertura de aristas:** 15/15 (100 %).
- **Cobertura de caminos principales:** 22/39.

Interpretacion.

- Los 10 casos son *suficientes para saturar EC*: cada decision del metodo termina ejercitandose por lo menos una vez (match al inicio, mismatch al medio, mismatch al final, patron mas largo que el sujeto, etc.).
- En cambio, no se alcanza PPC. Esto era esperable: PPC subsume EC pero no al reves. Los prime paths faltantes corresponden a combinaciones de iteraciones del `while`

externo y del `for` interno que esos diez casos puntuales no llegan a ejercitar (por ejemplo, patrones que `matchean` parcialmente en una posición intermedia, fallan y luego vuelven a `matchear` mas adelante).

- La instrumentación permite ver exactamente, por cada caso, que ruta del CFG se ejecuto y por que un prime path queda fuera: el reporte enumera todos los requisitos faltantes con su nombre canónico (lista de nodos).

Conclusiones

El práctico expone con claridad la jerarquía de subsunción $PPC \supseteq EPC \supseteq EC \supseteq NC$ y muestra, mediante ejemplos concretos, por que cada nivel “extra” aporta requisitos distintos:

- En el grafo del Ejercicio 1, NC e EC *coinciden* en la práctica porque la topología fuerza cada nodo a usar todas sus aristas. EC y EPC se distinguen recién con un test mínimo.
- En el grafo del Ejercicio 2 aparece la noción de *sidetrip*: un camino de test puede “visitar” un camino simple aunque no lo recorra de manera contigua, siempre que se vuelva al mismo nodo del que se desvió.
- Sobre código real (`fmtRerwrap` y `patternIndex`), los criterios estructurales funcionan como una guía para diseñar tests con *intención* y para medir, con herramientas como JaCoCo o con instrumentación propia, cuanto del comportamiento del programa queda efectivamente ejercitado. *Best Effort Touring* resulta especialmente útil cuando la semántica del programa hace infactibles algunos prime paths.